ISPilot Project - Root Level Work Checkpoint

Status: ☑ Production API validated; root services operational; business-scenario validation and Teams/Copilot integration remain the next gate
Date: 2026-08-29
API Endpoint: https://ispilot-api-46y2f3tyja-uc.a.run.app

Current Verified State

Root project

The root project remains the business logic layer: agent orchestration, domain services, analytics flows, and retrieval patterns from BigQuery. The multi-agent structure is in place and operational, with the coordinator delegating to specialized business services.

Verified status:

- Coordinator + specialized agents are part of the project design and execution flow
- Analytics and store services are integrated with the agent layer
- Root services remain the source of business logic and domain orchestration

API project

The API is the deployment layer that exposes the root logic through a FastAPI endpoint on Cloud Run. The production API is live and has passed a real bearer-token smoke test using the correct service-account impersonation pattern.

Verified status:

- Cloud Run service is responding at the production URL
- Authentication is working when using the correct Google identity token flow
- The service accepted a valid JSON request payload with `user_id`, `message`, and `session_id`
- No redeploy was needed after the payload/auth issue was corrected

Root cause of earlier issue

The earlier failure was not a root-service outage. It was caused by a combination of malformed request payload / shell quoting and then a separate GCP IAM impersonation issue while generating the bearer token for the production endpoint.

This is the key distinction to preserve in any handoff or future template reuse:

- the service is operational
- the contract is valid
- the issue was not the business logic or a broken deployment
- the real blockers were request formatting and token-generation permissions for the active Google identity

Validated remote auth flow

The working production validation path is now confirmed with the activated service account credentials:

```
# activate service account directly
gcloud auth activate-service-account \
  sa-tot-osa@corp-stro-salesinventory-prod.iam.gserviceaccount.com \
  --project=corp-stro-salesinventory-prod

TOKEN=$(gcloud auth print-identity-token)

curl -sS -X POST "https://ispilot-api-46y2f3tyja-uc.a.run.app/chat" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer ${TOKEN}" \
  -d '{"user_id":"debug-user","message":"Analiza la disponibilidad en góndola
de Talca Colín hoy","session_id":null}'
```

This returned a valid business answer with `status: "ok"`, including a real `session_id` and meaningful store-level operational analysis. This confirms the service is healthy end-to-end when using the correct identity context.

If the token generation fails with `PERMISSION_DENIED` / `iam.serviceAccounts.getAccessToken`, the next action is to grant `roles/iam.serviceAccountTokenCreator` to the impersonating identity or activate the correct service account directly before retrying the API call.

Completed work

Hardening and stability

- Auth validation for OAuth bearer flow and fallback API-key flow
- Request validation and payload handling
- Session handling and Firestore fallback behavior
- Error handling and structured responses
- Production docs and smoke-test guidance

Monitoring and observability

- Request and response logging
- Cloud Logging integration
- Metrics and latency tracking in API flow
- Operational visibility for production troubleshooting

Production validation

- Successful live validation of the Cloud Run endpoint with a valid bearer token
- Documented working command and usage pattern for future testing

---

Remaining work

1) Teams Bridge with Microsoft 365 Agents SDK (PRIORITY)

Goal: Create a proxy bot layer that sits between Microsoft Teams and ispilot-api, using the official Microsoft 365 Agents SDK (Python).

Architecture:

```
Teams User → Azure Bot Service
          ↓
      Teams Bridge Bot (Cloud Run)
      [MS 365 Agents SDK]
          ↓
      [JWT Validation from Azure]
          ↓
      ispilot-api (Cloud Run)
          ↓
      Vertex AI Reasoning Engine
```

Tasks:

- [ ] Register bot in Azure AD / Teams Developer Portal → get Microsoft App ID + Secret
- [ ] Create Python app with `microsoft-365-agents-sdk` + `fastapi`
- [ ] Implement `AgentAuthConfiguration` with JWT validation from Azure Bot Service
- [ ] Add handler to receive Teams activities and route to ispilot-api
- [ ] Deploy as new Cloud Run service (`teams-ispilot-bridge`)
- [ ] Configure `a365.config.json` with Microsoft App credentials and ispilot-api endpoint
- [ ] Test end-to-end: Teams message → Bridge → API → Response
- [ ] Add Microsoft Teams manifest and publish app to Teams catalog

Key Files to Create/Update:

- `teams_bot_bridge/a365.config.json` - Microsoft 365 Agents SDK config
- `teams_bot_bridge/main.py` - Bridge application entry point
- `teams_bot_bridge/handlers.py` - Message handlers
- `teams_bot_bridge/Dockerfile` - Cloud Run deployment
- `docs/TEAMS_SDK_INTEGRATION.md` - Complete setup guide

2) Business-scenario validation

Validate realistic retail questions across:

- store performance
- ranking and comparison questions
- daily summary flow
- inventory-relevant operational scenarios
- recommendation quality

3) Formal regression checks

Keep the documented bearer-token smoke test as the default regression check for deployment/auth updates.

Current decision

Current project status: production API is operational, the working smoke test is documented, and the next focus is real business validation and then Teams/Copilot integration.

Not currently required: another redeploy or service change based solely on the earlier malformed test payload.

Next milestone: formal validation of business use cases end-to-end, followed by channel integration into Teams/Copilot.

User Request Journey

```
1. User sends chat message via API
   POST https://ispilot-api.../chat
   {
     "user_id": "user123",
     "message": "How is Talca Colin performing?",
     "session_id": "session456"
   }

2. API Layer (ispilot-api)
   ├─ Validates auth (OAuth or API Key)
   ├─ Tracks request and latency
   ├─ Creates/loads session
   └─ Invokes Vertex agent layer

3. Root Layer (intelligent-shelf-agents)
   ├─ Coordinator receives the request
   ├─ Delegates to specialist logic
   ├─ Calls analytics/store services
   ├─ Queries BigQuery / data layer
   └─ Returns business answer

4. Response returns to API
   ├─ Session updated
   ├─ Logging and metrics recorded
   └─ JSON response delivered to client

5. Future extension
   ├─ Teams / Copilot front end
   ├─ API contract remains the stable backend surface
   └─ Business validation precedes broad user rollout
```

Execution plan

Phase A: Stabilize and document

- ☑ Working Cloud Run validation captured
- ☑ Correct message structure documented
- ☑ Auth flow documented
- ☑ No further deployment change required at this moment

Phase B: Business validation

- Validate real retail questions and actionability
- Check answer quality and routing
- Confirm session reuse / continuation works in production-style scenarios

Phase C: Teams / Copilot enablement

- Prepare connector or orchestration layer
- Expose validated API contract to enterprise channels
- Pilot with a small set of business scenarios before wider release

---

This checkpoint reflects the operational reality: the platform is ready for business validation and channel integration, not a fresh rebuild.